

Fundamentos: API de Claude, tool use y bucle agéntico — resumen de estudio

Bloque 0 · Versión 1.0 · 2026-08-05 · Guía oficial del examen v1.0 · CCAR-F

Eje → idea clave

EJE	TÍTULO	IDEA A RETENER
0.1	Anatomía Messages API	API stateless : el cliente reenvía el array <code>messages</code> completo en cada request; <code>system</code> es parámetro top-level, nunca un mensaje del array (salvo mid-conversation en Opus 5/4.8, Fable 5, Mythos 5, tras turno de usuario).
0.2	Definición de tools	<code>description</code> es el único canal para que el modelo sepa cuándo/cómo usar la tool; <code>strict: true</code> fuerza conformidad exacta del <code>input</code> con cualquier <code>tool_choice</code> .
0.3	Ciclo tool_use/tool_result	El bucle se controla con <code>stop_reason = "tool_use"</code> ; <code>tool_result</code> va inmediatamente después del <code>tool_use</code> y primero en el content array; server tools no llevan <code>tool_result</code> manual.
0.4	stop_reason	Único campo que decide si el bucle continúa o termina; verificarlo con <code>if/elif</code> explícito antes de tocar <code>content</code> , nunca inferir fin por lenguaje natural o contador de iteraciones.
0.5	tool_choice	4 valores (<code>auto</code> / <code>any</code> / tool forzada / <code>none</code>) gradúan libertad vs determinismo; <code>any</code> /forzada prefillan el mensaje del asistente (sin texto previo) e invalidan cache de bloques de mensaje.
0.6	Agente vs workflow	Decisión dirigida por el modelo (agente, <code>tool_choice: auto</code>) vs secuencia fija sin margen (workflow); para reglas no negociables, hook/gate programático — el prompt-only falla ~12% documentado.

Valores y enumeraciones memorizables

stop_reason → reacción

end_turn

Terminal, respuesta completa: usar el resultado tal cual.

tool_use

No terminal: ejecutar tool(s), enviar `tool_result`, continuar el bucle.

max_tokens

Truncada (posiblemente a media palabra): subir `max_tokens` o aceptar parcial.

stop_sequence

Se emitió una `stop_sequences` personalizada; campo `stop_sequence` indica cuál.

pause_turn

Límite de iteraciones internas de server tools (10 por defecto); reenviar el mensaje del asistente tal cual, sin `tool_result`, para continuar.

refusal

Rechazo por política; `stop_details` trae categoría (p. ej. "violence"); definir fallback, no reintentar a ciegas.

model_context_window_exceeded

Ventana llena (raro en modelos recientes); tratar como truncación.

tool_choice

auto (default)

Claude decide si llama a tool o responde en texto.

any

Debe llamar a alguna tool disponible, elige cuál; prefill automático.

{"type": "tool", "name": "..."}

Tool forzada exacta; prefill automático.

none

Solo texto, ninguna tool; default cuando no hay tools definidas.

Defaults, límites y restricciones

- `name` de tool: regex `^[a-zA-Z0-9_-]{1,64}$`; `description`: mínimo 3–4 oraciones.
- `input_examples` cuesta ~20–50 tokens/ejemplo simple, 100–200/complejo; **no soportado en server-side tools**.

- Orden estricto: `tool_result` inmediatamente tras el `tool_use` del asistente (nada interpolado); dentro del content array del mensaje, `tool_result` primero, texto adicional después — invertir produce error 400.
- Cada `tool_use` exige su `tool_result`; `tool_use_id` debe matchear exactamente.
- `strict: true` funciona con cualquier `tool_choice` (incluido `auto`); combinarlo con `any` da garantía compuesta (llamada + schema).
- Extended thinking **manual**: solo `tool_choice` `auto` / `none` (no `any` ni forzada); **adaptive** thinking sí soporta forzada.
- Cambiar `tool_choice` entre requests invalida bloques de mensaje cacheados (tools/system persisten cacheados).
- Límite de iteraciones internas de server tools antes de `pause_turn` : 10 por defecto.

Tabla de decisión comprimida

SITUACIÓN	ELECCIÓN	PORQUÉ
Salida estructurada garantizada	<code>tool_choice: any</code> + <code>strict: true</code>	Prefill fuerza la llamada; strict garantiza schema exacto
Orden crítico de pasos	Tool forzada en 1er paso, <code>auto</code> en el resto	Garantiza secuencia sin perder flexibilidad en lo no crítico
Conversación general, tool opcional	<code>auto</code> (default)	El modelo decide si la tool aporta valor
QA puro, sin efectos secundarios	<code>none</code>	Impide cualquier llamada a tool
Falla crítica no negociable (compliance, dinero)	Workflow determinista o hook, no solo prompting	Prompt-only falla ~12% documentado; el gate no depende del razonamiento del modelo
Tarea variable, requiere adaptación	Agente (<code>auto</code> , decisión del modelo)	Puede investigar y ajustar el plan dinámicamente
Tool ejecutada por Anthropic (<code>web_search</code> , <code>code_execution</code> ...)	Server tool: sin <code>tool_result</code> manual	Anthropic la resuelve internamente; enviarlo produce comportamiento indefinido
Tool ejecutada por tu código	Client tool: parsear, ejecutar, devolver <code>tool_result</code>	El bucle depende de que el cliente cierre el ciclo
Extended thinking manual activado	<code>tool_choice</code> solo <code>auto</code> / <code>none</code>	<code>any</code> /forzada no compatibles con thinking manual

Anti-patrones

- System message dentro del array** — `{"role": "system", ...}` produce error/comportamiento indefinido; `system` siempre top-level (salvo mid-conversation en los modelos excepcionados).
- Asumir memoria en servidor** — la API es stateless; no reenviar el historial completo pierde contexto. Enviar solo el último turno "para ahorrar tokens" causa el mismo fallo.
- Texto antes de tool_result en el content array** — invierte el orden requerido y provoca error 400; `tool_result` siempre primero.
- Ignorar stop_reason** — procesar `content` asumiendo siempre `end_turn` trata truncaciones (`max_tokens`), pausas (`pause_turn`) o rechazos (`refusal`) como respuestas finales.
- tool_result con mensaje de error genérico** — ("Operation failed") en vez de instructivo impide que Claude decida entre reintentar o escalar; además, omitir `is_error: true` le oculta el fallo por completo.
- Confiar solo en prompting para reglas críticas** — "verify customer first" como instrucción de texto falla ~12% de los casos documentado; usar hook o gate programático.

Trampas de examen

- `max_tokens` limita tokens de **salida**, no la ventana de contexto total.
- `pause_turn` (server tools aún en marcha, no terminal) vs `end_turn` (terminal): confundirlos entrega al usuario una respuesta incompleta.

Descripción insuficiente → *misrouting* (tool equivocada); schema insuficiente (campo `required` que no siempre llega) → alucinación de valores. Son dos fallos distintos con arreglos distintos.

`strict: true` funciona con **cualquier** `tool_choice`, incluido `auto` — no solo con `any` /forzada; lo que `any` añade es garantizar que se llame a una tool.

`tool_choice: any` es compatible con *adaptive thinking*, NO con extended thinking **manual**.

Cambiar `tool_choice` entre requests Sí invalida el cache de bloques de mensaje (tools/system quedan cacheados igualmente).

Un server tool nunca lleva `tool_result` manual del cliente; solo los client tools cierran el ciclo así.

Glosario mínimo

Client tool

Tool ejecutada por el código del cliente (user-defined o de schema Anthropic como `bash`, `text_editor`, `memory`, `computer`).

Server tool

Tool ejecutada internamente por Anthropic (`web_search`, `web_fetch`, `code_execution`, `tool_search`).

Prompt chaining

Secuencia fija de llamadas/pasos, pipeline determinista sin decisión del modelo.

Dynamic decomposition

El modelo genera subtareas a partir de hallazgos intermedios; dirigido por modelo.

Namespacing de tools

Prefijo por servicio (`github_list_prs`) para evitar confusión con muchas tools.

Consolidación de tools

Agrupar en pocas tools con parámetro `action` en vez de muchas hiperespecíficas.

Grammar-constrained sampling

Mecanismo detrás de `strict: true` que restringe el muestreo al schema exacto.

Coordinador-subagente

Patrón dirigido por modelo: el coordinador decide qué subagentes invocar y en qué orden.